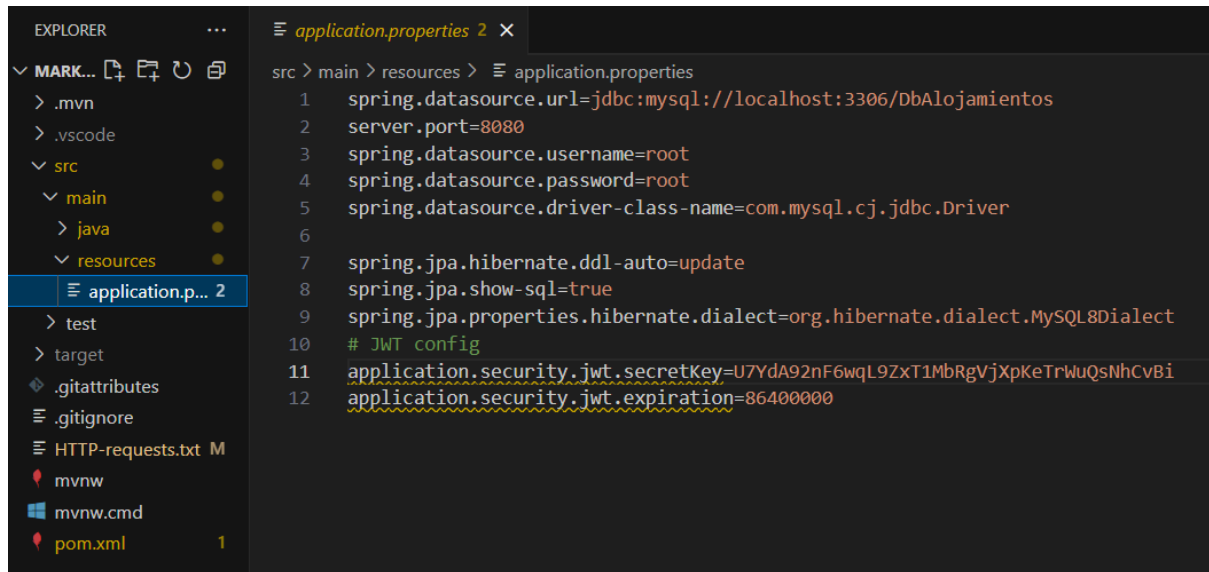


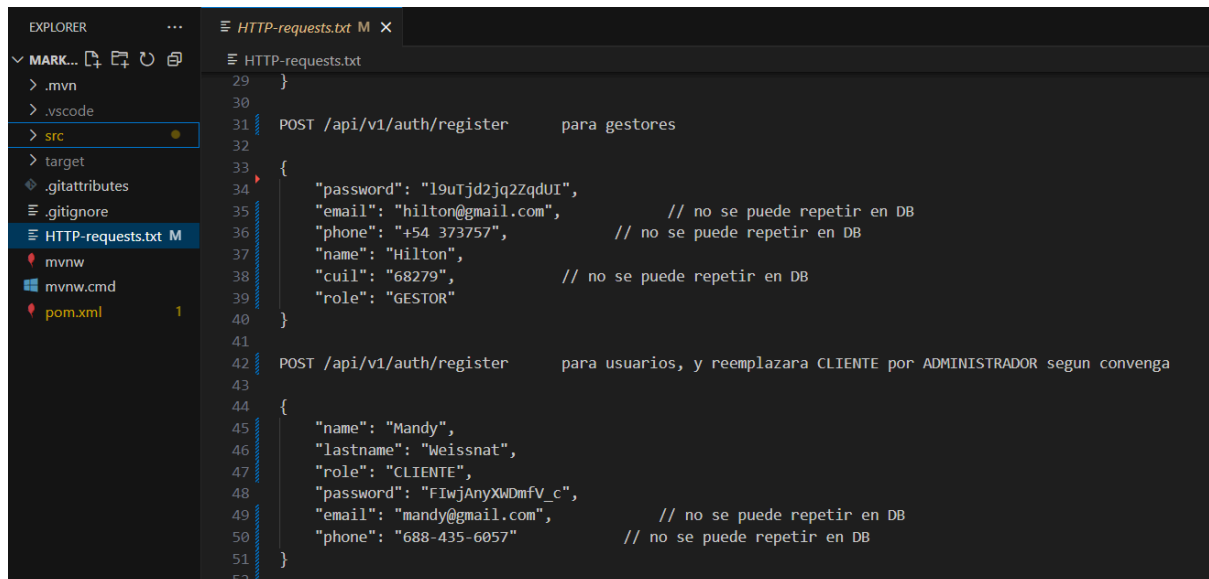
Primero que nada asegurarse de tener MySQL ejecutandose en localhost:3306 y tener creada una base de datos llamada como tengamos en el application.properties y que el usuario y contraseña sean como pongamos

A screenshot of the Visual Studio Code editor. The Explorer sidebar on the left shows a project structure with folders like .mvn, .vscode, src, main, java, and resources. The 'application.p...' file in the resources folder is selected. The main editor area shows the content of 'application.properties' with the following properties:

```
src > main > resources > application.properties
1  spring.datasource.url=jdbc:mysql://localhost:3306/DbAlojamientos
2  server.port=8080
3  spring.datasource.username=root
4  spring.datasource.password=root
5  spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
6
7  spring.jpa.hibernate.ddl-auto=update
8  spring.jpa.show-sql=true
9  spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
10 # JWT config
11 application.security.jwt.secretKey=U7YdA92nF6wqL9ZxT1MbRgVjXpKeTrWuQsNhCvBi
12 application.security.jwt.expiration=86400000
```

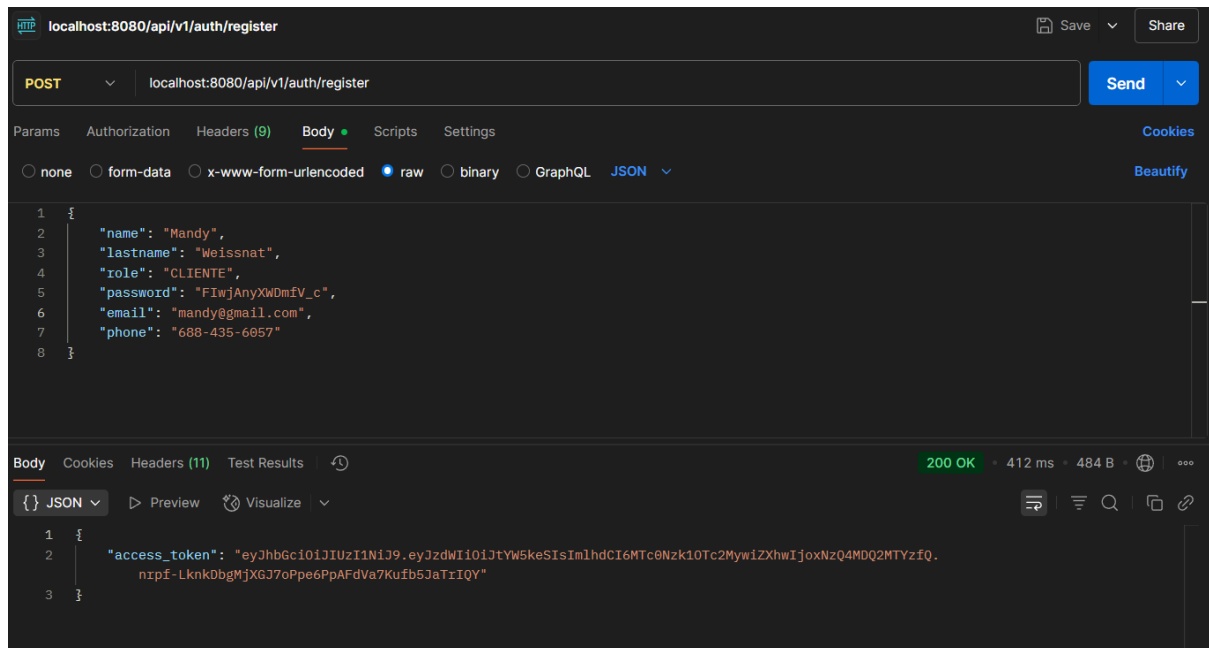
Deberia verse asi, reemplazando con sus datos

Lo segundo es usar desde el archivo HTTP-requests.txt que esta en la raíz alguno de los ejemplos de POST que hay, para usar los endpoints, excepto el get de /api/v1/categorias y los post /api/v1/auth/**, hay que estar autenticado asi que si no tienen cuenta creada haganla usando cualquiera de los 2 post:

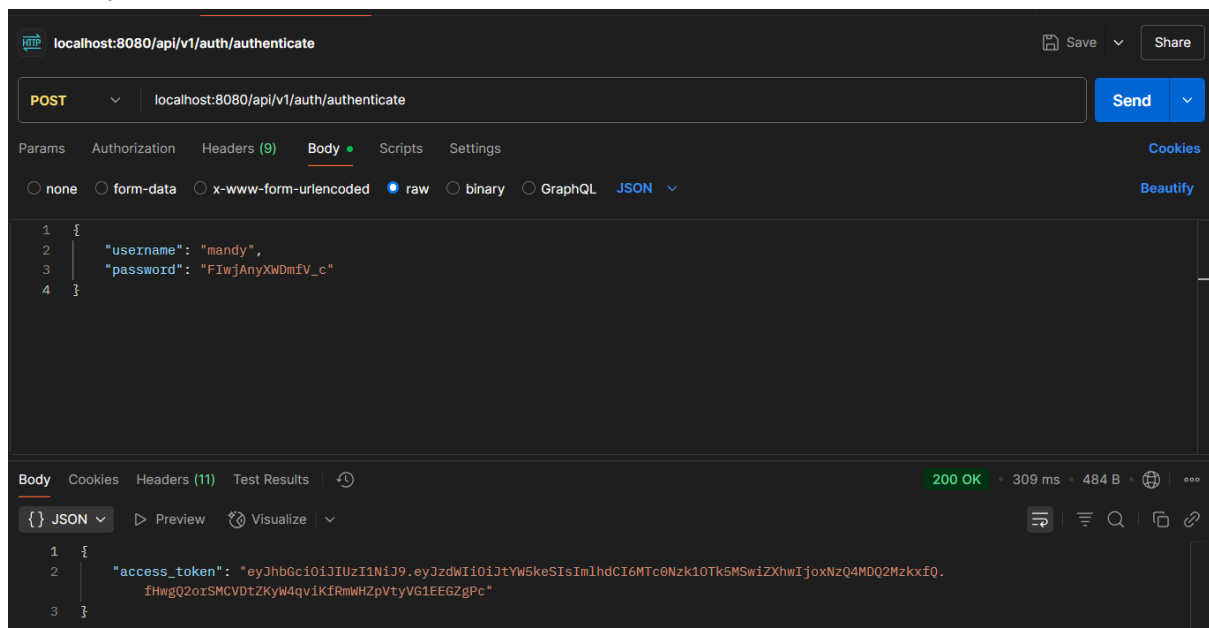
A screenshot of the Visual Studio Code editor showing the 'HTTP-requests.txt' file. The Explorer sidebar on the left shows the file is located in the root of the project. The main editor area displays two POST requests for the /api/v1/auth/register endpoint:

```
EXPLOREDER  ...  HTTP-requests.txt M X
HTTP-requests.txt
29  }
30
31  POST /api/v1/auth/register      para gestores
32
33  {
34      "password": "l9uTjd2jq2ZqdUI",
35      "email": "hilton@gmail.com",           // no se puede repetir en DB
36      "phone": "+54 373757",                // no se puede repetir en DB
37      "name": "Hilton",
38      "cuil": "68279",                     // no se puede repetir en DB
39      "role": "GESTOR"
40  }
41
42  POST /api/v1/auth/register      para usuarios, y reemplazara CLIENTE por ADMINISTRADOR segun convenga
43
44  {
45      "name": "Mandy",
46      "lastname": "Weissnat",
47      "role": "CLIENTE",
48      "password": "FIWjAnyXwDmfV_c",
49      "email": "mandy@gmail.com",           // no se puede repetir en DB
50      "phone": "688-435-6057"              // no se puede repetir en DB
51  }
52
```

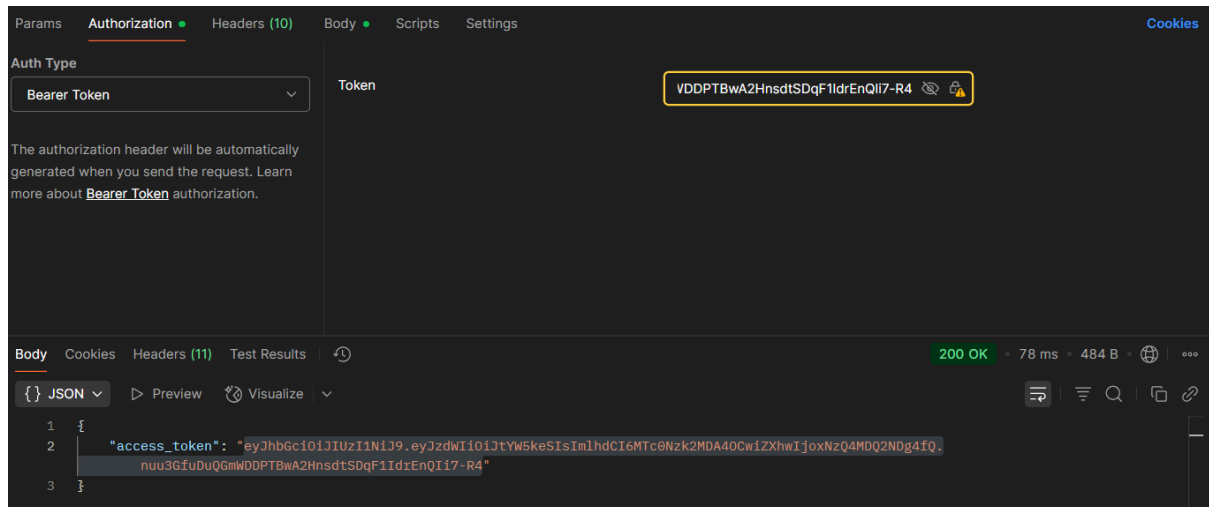
Después de pegar en post man en ese http y con el method puesto en POST, tocar send y copiar el access token:



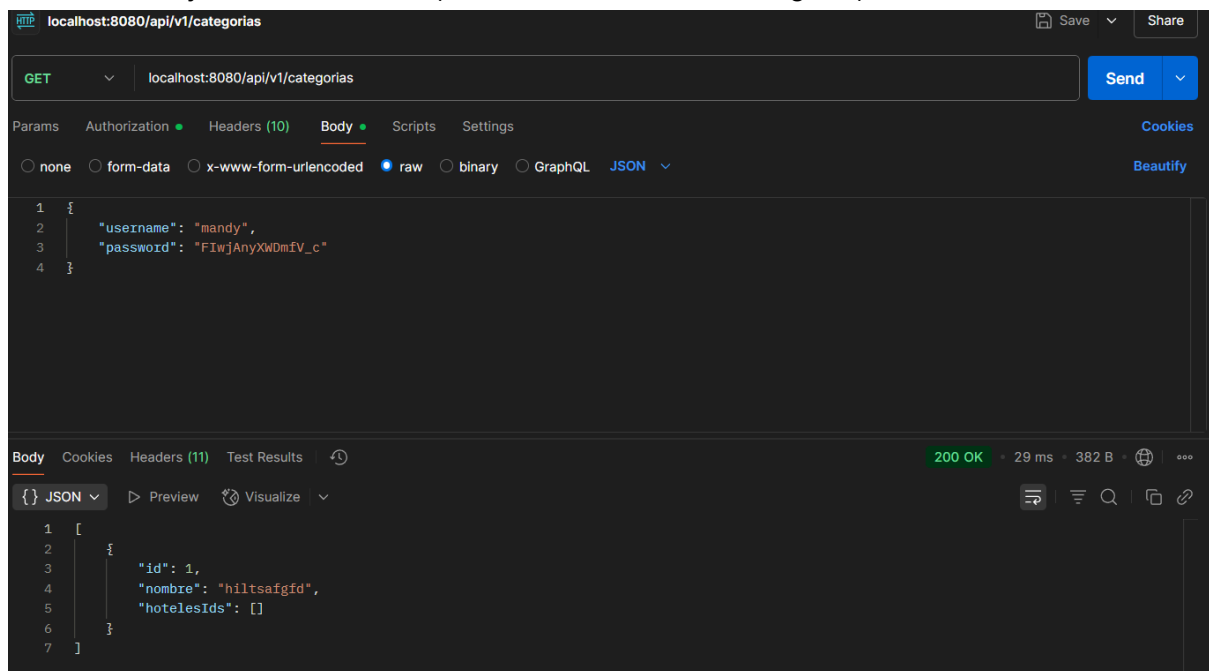
Y a partir de ahora como se creo en la base de datos ya se puede usar el login solo mandando username(que se pone automáticamente todo lo que había antes del arroba en el mail) y la password:



Ahora copiando y pegando el token en la pestaña de authorization y con el auth type en Bearer Token ya se considera que estas logueado con esa cuenta y tenes acceso a todo lo authenticated y lo que el rol permita:



Por ejemplo esta cuenta que es cliente me va a dejar hacer un GET de las categorías, pero no me va a dejar hacer un POST(osea crear una nueva categoría):



(nótese que en GET no importa lo que haya en el body)

localhost:8080/api/v1/categorias

SaveShare

POST

localhost:8080/api/v1/categorias

Send

Params

Authorization

Headers (10)

Body

Scripts

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Beautify

1 {

2 | "nombre": "mandy"

3 }

Body

Cookies

Headers (10)

Test Results

403 Forbidden

17 ms

300 B

Raw

Preview

Visualize

1